

EmporiUm of Connecticut &
Florida

Annual Review

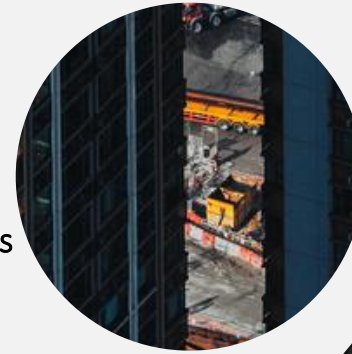
Lana Ilana and
Ellen Lemon

May 29, 2026

Data Analytics Team

Agenda

- Who are the territory managers for the sales territories assigned? What are the store IDs and cities for the stores in each assigned sales territory?
- What is the monthly total revenue for in-store sales in each of the two sales territories, over the full period covered by the data?
- How would you rank the sales performance of each store in each sales territory? Which are the top-performing stores?
- Comparing the customer ID from the customer list data with the rewards ID from the sales data, who were the top customers in each sales territory?
- What is the number of transactions per month by product category in each assigned territory? What is the total sales revenue per month by category? What might this tell you about the most popular products, and where could there be an opportunity for growth?
- What is your recommendation for where to focus marketing attention in the next quarter?





Researching my territory

Let's dive in



SALES TERRITORY INFO

Core Marketing Analysis

Q1: Who are the territory managers for the sales territories assigned? What are the store IDs and cities for the stores in each assigned sales territory?

The territory managers for my sales territories assigned are Ellen Lemon & Lana Ilana. The store IDs are listed from 865 down to 729. There are 19 different cities for the stores assigned to the sales territory together. Connecticut with 8 and Florida with 11.

```
[8]: Territory_managers = StoreDetail[
      StoreDetail["State"].isin(["Florida", "Connecticut"])
      ]
Territory_managers.head()
```

```
[8]:
```

	Store Location	State	Store ID	Territory Manager	Region	Region Director
18	Bridgeport	Connecticut	865	Ellen Lemon	East	Ana Jilani
19	Darien	Connecticut	866	Ellen Lemon	East	Ana Jilani
20	Hartford	Connecticut	867	Ellen Lemon	East	Ana Jilani
21	Litchfield County	Connecticut	868	Ellen Lemon	East	Ana Jilani
22	New Haven	Connecticut	869	Ellen Lemon	East	Ana Jilani

MONTHLY SALES REVENUE

How are sales doing month by month? Have they changed?

```
Sales = pd.merge(
    StoreSales,
    Territory_managers,
    on="Store ID",
    how= "inner"
)
# This only merges the two stores, which only gives us the first step of
# answering the question.

Sales["Transaction Date"] = pd.to_datetime(Sales["Transaction Date"])
# Converting a datetime column

Sales["Month"] = Sales["Transaction Date"].dt.to_period("M")
# Makes a month column

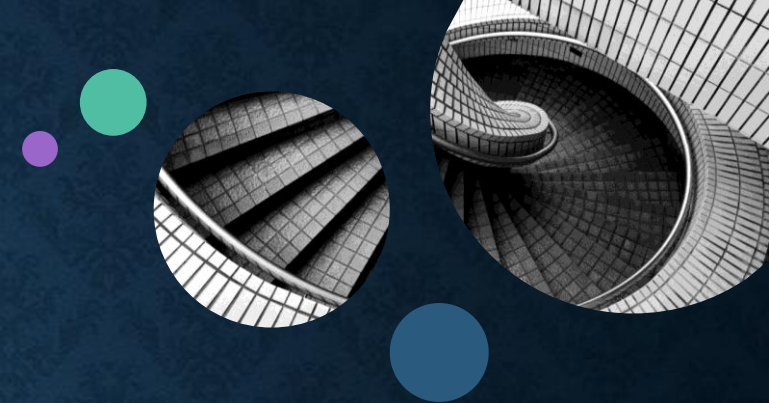
Sales["Rank"] = Sales.groupby(
    "State")["Sale Amount"].rank(
    ascending=False)

monthly_revenue = Sales.groupby(
    ["State", "Month"])["Sale Amount"].sum().reset_index()
# By using .reset_index, it converts the series back into a DataFrame
print(monthly_revenue)
```

	State	Month	Sale Amount
0	Connecticut	2022-01	29958.25
1	Connecticut	2022-02	25563.98
2	Connecticut	2022-03	33995.57
3	Connecticut	2022-04	31587.12
4	Connecticut	2022-05	37953.48
..	...	...	...
91	Florida	2025-08	129936.53
92	Florida	2025-09	106935.33
93	Florida	2025-10	188215.96
94	Florida	2025-11	133022.84
95	Florida	2025-12	122408.27

[96 rows x 3 columns]

RANKINGS!



```
store_performance = Sales.groupby(
    ["State", "Store ID", "Store Location"])["Sale Amount"].sum().reset_index()
# By doing this, we know the total sales for each store.

store_performance["Rank"] = store_performance.groupby(
    "State")["Sale Amount"].rank(
    ascending=False) # ranks stores within each territory

store_performance = store_performance.sort_values(
    ["State", "Rank"]
)
# This sorts the results of the ranking

print(store_performance)
# Shows each store total revenue ranking within the territories Connecticut and Florida.
# As we can see for Connecticut, the top store is Store 870 in New London with
# 331,116.76.
# Florida top ranking store is store 724 in Miami with 618,846.25.
```

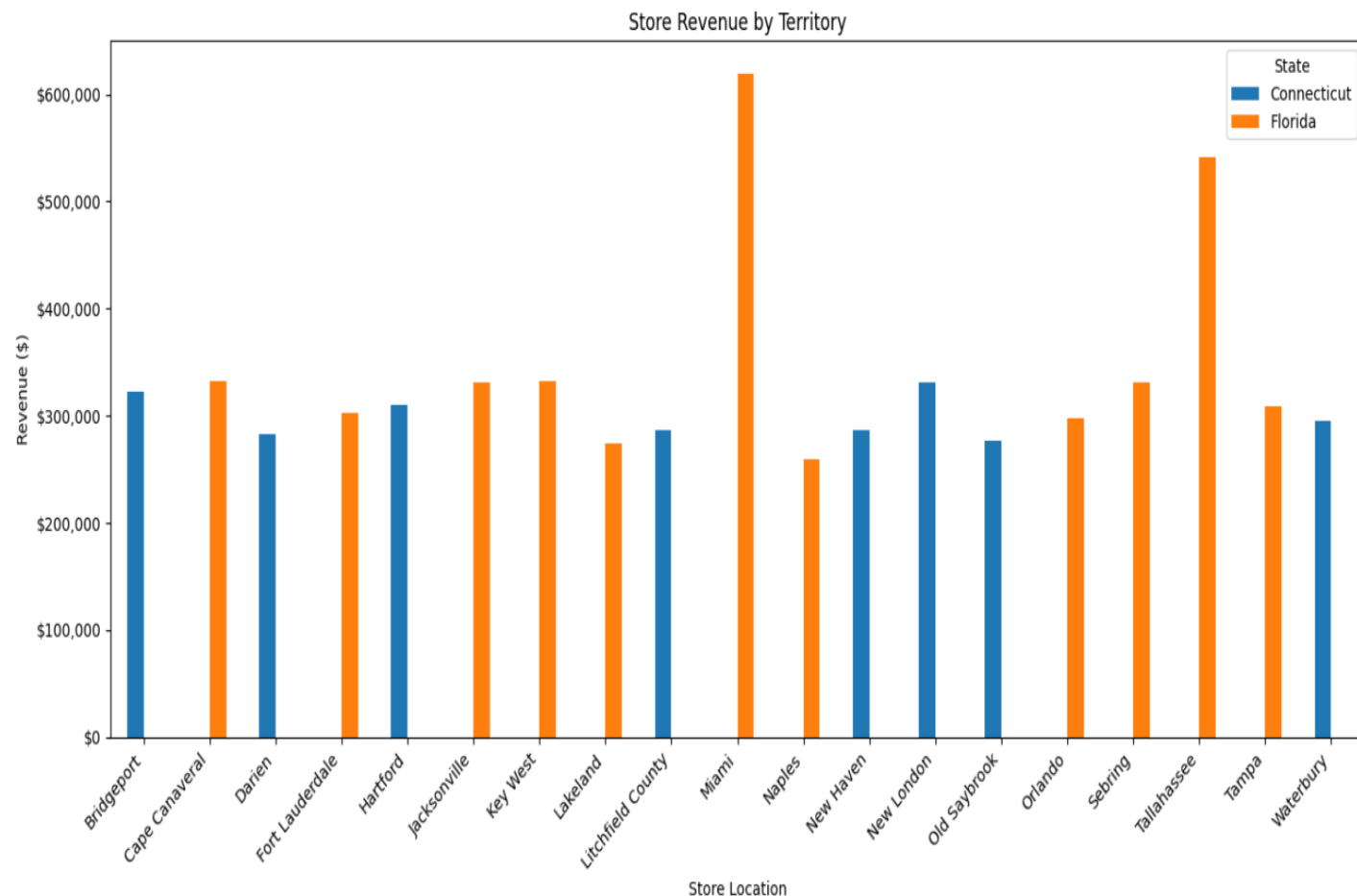
	State	Store ID	Store Location	Sale Amount	Rank
5	Connecticut	870	New London	331116.76	1.0
0	Connecticut	865	Bridgeport	322100.07	2.0
2	Connecticut	867	Hartford	309592.06	3.0
7	Connecticut	872	Waterbury	295651.17	4.0
4	Connecticut	869	New Haven	286779.43	5.0
3	Connecticut	868	Litchfield County	286217.15	6.0
1	Connecticut	866	Darien	283455.62	7.0
6	Connecticut	871	Old Saybrook	277310.18	8.0
13	Florida	724	Miami	618846.25	1.0
17	Florida	728	Tallahassee	541670.66	2.0
11	Florida	722	Key West	332620.43	3.0
8	Florida	719	Cape Canaveral	331956.13	4.0
10	Florida	721	Jacksonville	331667.08	5.0
16	Florida	727	Sebring	330848.14	6.0
18	Florida	729	Tampa	308979.01	7.0
9	Florida	720	Fort Lauderdale	302270.07	8.0
15	Florida	726	Orlando	297891.71	9.0
12	Florida	723	Lakeland	273835.24	10.0
14	Florida	725	Naples	259602.83	11.0

VISUALIZE IT!

Coding

```
pivot_table = store_performance.pivot_table(  
    index="Store Location",  
    columns="State",  
    values="Sale Amount",  
    aggfunc="sum"  
)  
  
ax = pivot_table.plot(  
    kind="bar",  
    figsize=(14, 7)  
)  
  
plt.title("Store Revenue by Territory")  
plt.xlabel("Store Location")  
plt.ylabel("Revenue ($)")  
  
plt.xticks(rotation=45, ha='right')  
  
ax.yaxis.set_major_formatter(  
    StrMethodFormatter('${x:,.0f}')  
)  
  
plt.tight_layout()  
  
plt.show()
```

Bar Chart



TOP CUSTOMERS!

```
customer_sales = pd.merge(
    customer_list, Sales,
    left_on="cust_id",
    right_on="RewardsID",
    how="inner") # This merges the tables and matches the keys, keeping only the matching customers.
```

```
top_customers = customer_sales.groupby(
    ["State", "cust_id", "name"])["Sale Amount"].sum().reset_index()
# Here we are grouping the table by the territory and customers. using this we can
# calculate the total spendings of each customer. The reason we don't group by store
# location is because it will mess with the ranking system. A customer could be shown
# twice because they go to different stores.
```

```
top_customers["Rank"] = top_customers.groupby(
    "State")["Sale Amount"].rank(
    ascending=False, method="dense")
```

```
top_customers[top_customers["Rank"] == 1]
# This is where the ranking of the customers comes in.
```

```
top_customers = top_customers.sort_values(
    ["State", "Rank"])
# Always make sure to sort the results; it makes the ranking clear. It will show the
# best performers first.
top_customers
# The top customer in the sales territory of Connecticut is Marie S. with $4,338.87.
# The top customer in the sales territory of Florida is Harry Morgan with $19.87.
```

	State	cust_id	name	Sale Amount	Rank
17	Connecticut	18	Marie S.	4388.87	1.0
20	Connecticut	21	Jon Snow	3676.21	2.0
311	Connecticut	322	Fox Mulder	3569.60	3.0
351	Connecticut	365	Pugsley A.	3009.29	4.0
98	Connecticut	104	Helen M.	2886.72	5.0
...	...	...	...	...	...
732	Florida	234	Harry Morgan	19.87	515.0
766	Florida	268	Lindsay F.	12.30	516.0
968	Florida	470	Bubbles	9.45	517.0
1006	Florida	508	Gary	9.09	518.0
526	Florida	27	J. Lannister	5.00	519.0

1020 rows × 5 columns

CATEGORY TRANSACTIONS & MONTHLY REVENUE

```
Sales_products = pd.merge(
    Sales,
    Products,
    on="Prod Num")

Sales_category = pd.merge(
    Sales_products,
    ProductCategories,
    on="CategoryID")

# Since Sales doesn't have anything to match with Category, we have to merge twice. First
# with Products to get the CategoryID for us to merge with Category to get the Categories.

Sales_category["Transaction Date"] = pd.to_datetime(
    Sales_category["Transaction Date"])
# Since the question is asking for the month, we must convert the column of dates first.

Sales_category["Month"] = Sales_category["Transaction Date"].dt.to_period("M")
# This creates the month column the question wants per month specifically. dt refrencing
# the datetime and ("M") representing month.

transactions_per_month = Sales_category.groupby(
    ["State", "Month", "Category"])[["CategoryID"].count().reset_index()
# We use the count function so we can answer the question of how many transactions per
# month.
transactions_per_month = transactions_per_month.rename(
    columns={"CategoryID": "Transaction Count"})
# This answers the number of transactions per month

transactions_per_month = transactions_per_month.sort_values(
    ["State", "Month", "Category"])
# This sorts the results
```

	State	Month	Category	Transaction Count
0	Connecticut	2022-01	Apparel and Merchandise	336
1	Connecticut	2022-01	Art Supplies	319
2	Connecticut	2022-01	Books (General)	76
3	Connecticut	2022-01	Stationery and Supplies	396
4	Connecticut	2022-01	Technology & Accessories	315
...	...	...	...	...
571	Florida	2025-12	Art Supplies	1507
572	Florida	2025-12	Books (General)	284
573	Florida	2025-12	Stationery and Supplies	1958
574	Florida	2025-12	Technology & Accessories	1344
575	Florida	2025-12	Textbooks	957

576 rows x 4 columns

PART TWO MONTHLY CATEGORY REVENUE

```
monthly_category_revenue = Sales_category.groupby(  
    ["State", "Month", "Category"] ["Sale Amount"].sum().reset_index()  
monthly_category_revenue
```

	State	Month	Category	Sale Amount
0	Connecticut	2022-01	Apparel and Merchandise	9786.00
1	Connecticut	2022-01	Art Supplies	7184.32
2	Connecticut	2022-01	Books (General)	1958.40
3	Connecticut	2022-01	Stationery and Supplies	3679.39
4	Connecticut	2022-01	Technology & Accessories	153020.42
...	...	...	...	...
571	Florida	2025-12	Art Supplies	44036.85
572	Florida	2025-12	Books (General)	9101.36
573	Florida	2025-12	Stationery and Supplies	19880.08
574	Florida	2025-12	Technology & Accessories	652799.77
575	Florida	2025-12	Textbooks	166766.16

576 rows × 4 columns

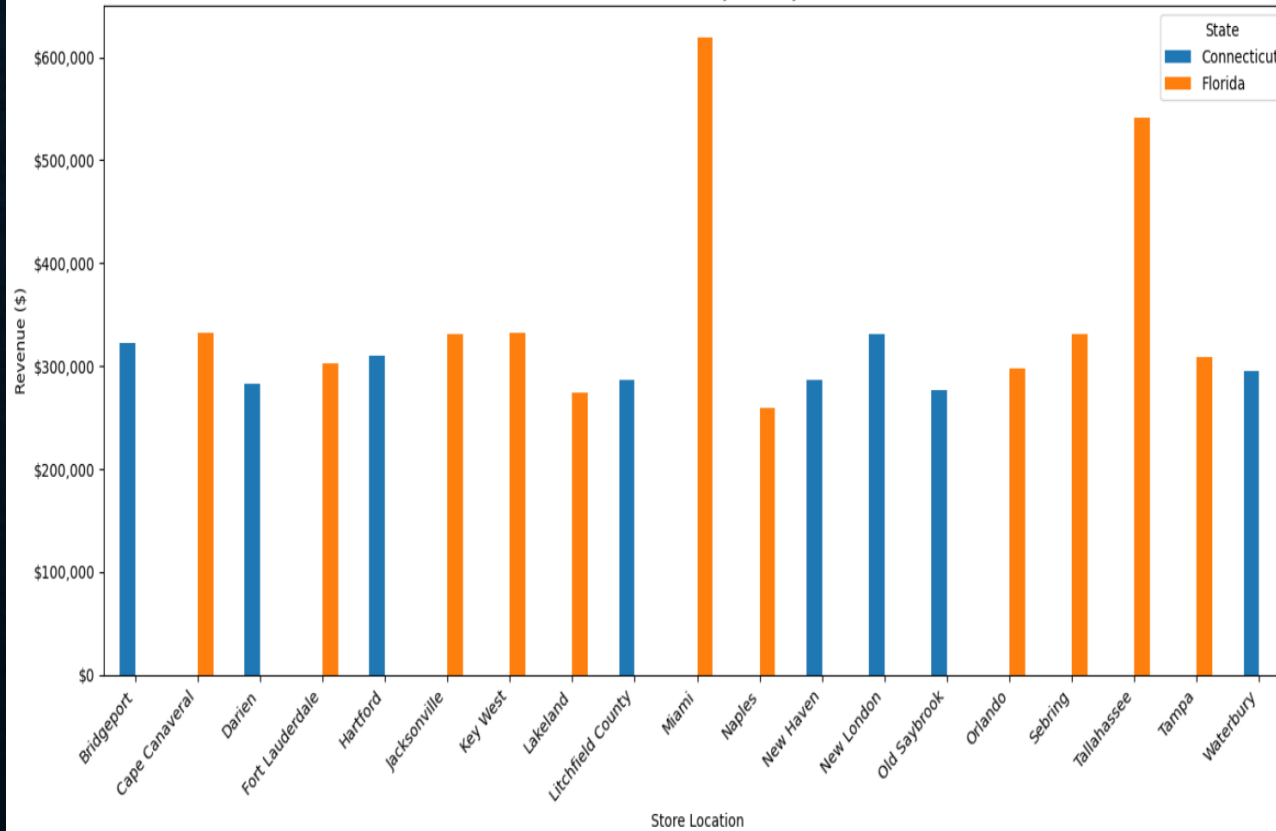
```
pivot_table = monthly_category_revenue.pivot_table(  
    index="Category",  
    columns="State",  
    values="Sale Amount",  
    aggfunc="sum"  
)  
  
ax = pivot_table.plot(  
    kind="bar",  
    figsize=(12, 6)  
)  
  
plt.title("Monthly Sales Revenue by Category")  
plt.xlabel("Category")  
plt.ylabel("Sale Amount ($)")  
plt.xticks(rotation=30, ha='right')  
  
ax.yaxis.set_major_formatter(  
    StrMethodFormatter('${x:,.0f}')  
)  
  
plt.tight_layout()  
plt.show()
```

Monthly Sa
10

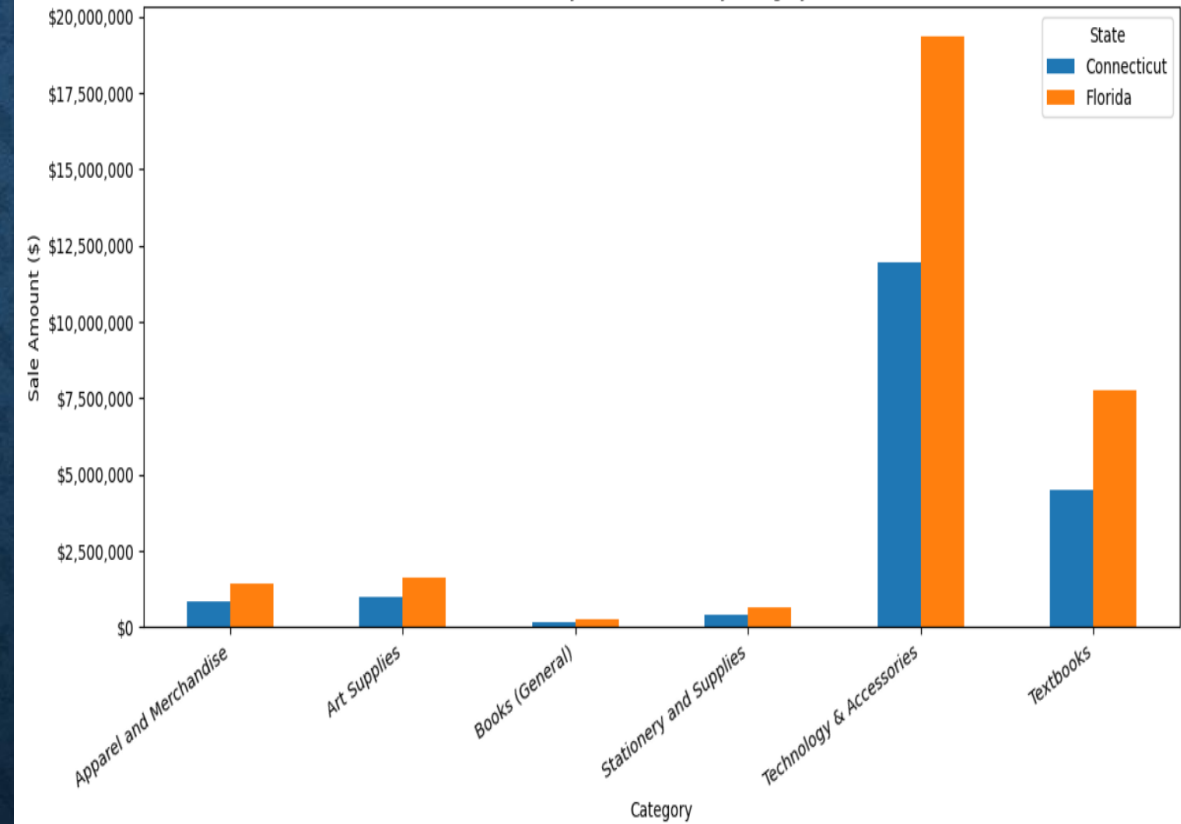
RECOMMENDATIONS



Store Revenue by Territory



Monthly Sales Revenue by Category



Conclusion

Thanks for your time and attention.

I take pride in my work and hope you consider my recommendations.

Looking forward to hearing your follow-up on the coming new year's sales.

Sha'Rya Weaver

https://github.com/Sharya-wolf/Capstone_2.git

Data Analytics

